LegalEase: AI-Powered Legal Document Generator

Project Documentation

A web application for AI-assisted drafting, review, and management of legal documents.

TEAM MEMBERS	EMAIL ID
Praveen kumar S	praveenks13112007@gm
Pradeep G	pradeep03122007gp@gr
Poomalai M	poomalai.m2008@gmail
Nithish V	vnithishcs28@gamil.c
Naveen M	naveen.manics@gma

(Name slots left blank for the team to fill in)

INTRODUCTION

Project Title: **LegalEase – AI-Powered Legal Document Generator**

PROJECT OVERVIEW

LegalEase is an AI-powered web application that helps individuals, small businesses, and legal professionals generate, review, and manage legal documents quickly and accurately. It combines a curated library of legal templates with a generative AI drafting engine, guiding users through a structured intake process and producing a polished, editable document in minutes.

PURPOSE

LegalEase aims to make legal document creation faster, more affordable, and less error-prone by pairing trusted templates with AI-assisted drafting. The main goals are:

- Accessible Legal Drafting – Guided, plain-language intake forms that turn user answers into properly structured legal language.
- AI-Assisted Accuracy – Large language model drafting combined with clause libraries to reduce errors and omissions.
- Modern Tech Stack – Built with React.js, Node.js/Express, and a large language model API for fast, reliable generation.

FEATURES

- AI Document Generation – Generates first-draft legal documents (NDAs, leases, contracts, policies) from a short questionnaire.
- Template Library – Browsable, categorized library of attorney-reviewed starting templates.
- Clause Editor – In-browser rich text editor for reviewing, editing, and annotating generated clauses.
- Smart Search – Keyword and category search across templates and saved documents.
- Export & E-Signature – Export to PDF/DOCX and route documents for e-signature.
- Version History – Tracks edits and lets users revert to a previous draft.

ARCHITECTURE

Component Structure

The application is divided into three main sections:

- Pages – Full-page components (Home, Template Library, Document Editor, Dashboard).
- Components – Reusable UI elements (Navbar, Questionnaire, Document Card, Clause Panel).
- Styles – CSS/Tailwind styling files.

State Management

- Global State: Managed using the React Context API for the active document and user session.
- Local State: Controlled via React's useState for component-level interactions (form inputs, modals).

Routing: Implemented using React Router to enable seamless navigation between pages.

SETUP INSTRUCTIONS

Prerequisites

- Node.js & npm – Install from the Node.js website.
- React.js – Set up a new project using: `npx create-react-app legalease-app`

```
cd legalease-app npm start
```

Installation Steps

1 Clone the repository:

```
git clone https://github.com/example-org/LegalEase-AI-Document-Generator
```

Navigate into the project directory `cd legalease-app`

1 Install dependencies:

```
npm install
```

1. Set up environment variables by creating a .env file and adding the necessary API keys (e.g. AI provider key, e-signature provider key).
2. Start the development server:

```
npm start
```

Access the application

Open `http://localhost:3000` in your web browser.

FOLDER STRUCTURE

The project is structured into different directories for better organization and maintainability. Below is an overview of the folder structure.

Client

The client folder (inside src/) contains the core files of the frontend application. It consists of:

- Components: Houses all reusable UI components such as buttons, cards, and the navigation bar.
- Pages: Contains full-page components, including the homepage, template library, and document editor.
- Styles: Includes all CSS or SCSS files used to style the application.
- Assets: Stores images, icons, and other static files used throughout the app.

```
src/ components/ Navbar.js
Hero.js
TemplateCard.js
Questionnaire.js ClauseEditor.js pages/ Home.js
TemplateLibrary.js
DocumentEditor.js Dashboard.js context/ DocumentContext.js styles/ assets/
```

RUNNING THE APPLICATION

To start the frontend server, run:

```
npm start
```

Then, open `http://localhost:3000` in your browser.

COMPONENT DOCUMENTATION

KEY COMPONENTS

Below are the major components of the LegalEase application, along with their purpose and props:

Navbar Component (Navbar.js)

Purpose:

- Provides site-wide navigation.
- Contains links to Home, Template Library, Dashboard, and Account.

Props:

- logo (string): Path to the logo image.
- menuItems (array): List of menu items for navigation.

Hero Component (Hero.js)

Purpose:

- Displays an introduction to the application.
- Contains a call-to-action button to start generating a document.

Props:

- title (string): Main heading text.
- subtitle (string): Supporting description text.
- buttonText (string): Label for the action button.

Template Card Component (TemplateCard.js)

Purpose:

- Displays a brief summary of a legal template, including an icon, title, and category.
- Redirects users to the guided questionnaire for that template when clicked.

Props:

- template (object): Contains id, title, category, and description.

Questionnaire Component (Questionnaire.js)

Purpose:

- Collects structured user input used to populate the AI-generated document.
- Validates required fields before submission.

Props:

- templateId (string): ID of the selected template.
- onSubmit (function): Callback fired with the collected answers.

Document Editor Component (ClauseEditor.js)

Purpose:

- Displays the AI-generated draft with editable clauses.
- Highlights AI-suggested edits and lets users accept, reject, or rewrite clauses.

Props:

- documentId (string): Unique ID used to fetch the generated draft.

REUSABLE COMPONENTS

Search Bar Component (SearchBar.js)

Purpose:

- Allows users to search templates and saved documents by keyword.

Props:

- onSearch (function): Callback function for handling search queries.

Button Component (Button.js)

Purpose:

- A reusable button component with customizable styles.

Props:

- text (string): Button label.
- onClick (function): Click event handler.
- variant (string): Defines button styles (e.g., primary, secondary).

Loading Spinner Component (Loading.js)

Purpose:

- Displays a loading animation while the AI drafts or refines a document.

Props:

- size (string): Size of the spinner (small, medium, large).

STATE MANAGEMENT

State management in the LegalEase application ensures efficient data handling across different components. The project incorporates both global state for shared data and local state for component-specific interactions.

GLOBAL STATE

The application uses the React Context API to manage global state, allowing components to access shared data without prop drilling.

Global State Usage in LegalEase:

- Document Data Storage: Stores the active document, its clauses, and generation status.
- Navigation State: Manages the active template category and filters.
- User Preferences: Keeps track of saved documents and favorite templates.

How global state flows across the application:

- The DocumentContext.js file initializes and manages the global state.
- The DocumentProvider component wraps the application to provide access to the global state.
- Components like TemplateLibrary.js and ClauseEditor.js consume global state using useContext.
- Any updates to the global state (e.g., a new AI-generated draft) automatically reflect across all dependent components.

LOCAL STATE

Local state is managed using the useState hook within individual components. It is used for handling temporary UI interactions that don't need to persist across multiple components.

Local State Usage in LegalEase:

- Search Bar: Stores and updates the user's search input dynamically. •Questionnaire: Keeps track of the current step and entered answers.
- UI Components: Controls modals, dropdown menus, and loading states.

How Local State Works in Components:

- The component initializes a state variable using useState().
- State updates dynamically based on user interactions.
- Changes to local state trigger a re-render of the specific component without affecting others.

USER INTERFACE

The LegalEase application features a modern and intuitive user interface, designed to provide a seamless experience for users generating and managing legal documents. Below are key UI elements along with their descriptions.

Home Page (Hero Section & Search)

Features:

- A welcome banner introducing the app.
- A search bar allowing users to quickly find a document template.
- A call-to-action button for starting a new document.

Template Library Page

Features:

- Displays various legal document categories (Business, Real Estate, Employment, Personal).
- Each template card includes an icon and title.
- Clicking a template starts the guided questionnaire.

Document Editor Page

Features:

- Shows the AI-generated document with clause-by-clause editing.
- Includes a save-to-dashboard option.
- Responsive layout for mobile and desktop users.

Dashboard Section

Features:

- Displays the user's saved and in-progress documents.
- Each document card includes a title, status, and last-edited date.
- Clicking a document reopens it in the editor.

Export & E-Signature Form

Features:

- Users can export a finished document as PDF or DOCX.
- Clean and simple form to send a document out for e-signature.

Responsive UI & Mobile View

Features: The application is fully responsive across different screen sizes.
•Mobile-friendly navigation menu and document cards.

STYLING

LegalEase is designed with a modern, responsive, and clean aesthetic using industry-standard styling techniques.

CSS Frameworks & Libraries Used

Tailwind CSS / Bootstrap:

- Tailwind CSS is a utility-first framework that allows for highly customized and flexible styling, reducing the need for external stylesheets.
- Bootstrap provides a grid system and predefined components where rapid layout is needed.
- The combination of these frameworks ensures consistent layouts across all screen sizes (mobile, tablet, desktop) and faster development through pre-built, reusable components.

React Icons:

- Used for UI elements such as search icons, navigation arrows, status badges, and buttons.

Theming & Custom Design

Consistent Color Scheme:

- The application follows a uniform, professional color palette for a clean and readable UI.
- Colors are chosen to provide good contrast and accessibility.
- Primary Color: Used for buttons and highlights.
- Secondary Color: Used for backgrounds and supporting elements.
- Text Color: Optimized for readability.

Dark Mode & Light Mode:

- Dark Mode is implemented to provide a comfortable experience during long editing sessions.
- Users can toggle between Light Mode (default) and Dark Mode.
- Future Enhancement: Saving the user's preference in local storage to persist across sessions.

Custom CSS Modules & Styled Components:

- CSS Modules scope styles to specific components, preventing unwanted global style conflicts.
- Styled Components allow writing CSS directly within JavaScript files and enable dynamic styling based on props (e.g., different styles per document status).

Animations & Transitions

Smooth UI Interactions:

- CSS animations and libraries such as Framer Motion create a seamless drafting experience.
- Hover effects on buttons and cards indicate interactivity.
- Fade-in animations for AI-generated content appearing dynamically.
- Slide transitions when navigating between questionnaire steps.

Performance Optimization:

- Avoiding excessive animations to keep the app lightweight.
- Using lazy loading for images and content to enhance page load speed.

TESTING

Ensuring the reliability and accuracy of the application is critical for a legal-adjacent product. The testing approach includes multiple testing strategies.

Testing Strategy

Unit Testing:

- Used Jest and React Testing Library to test individual components.
- Ensured that components render correctly and respond to user actions appropriately.
- Tested UI components such as buttons, forms, and the questionnaire to verify expected behavior.

Integration Testing:

- Verified that AI generation calls return and render a complete draft document.
- Ensured that selecting a template updates the questionnaire and preview correctly.
- Tested data flow between components to validate overall functionality.

End-to-End Testing:

- Used Cypress to simulate real user interactions and validate complete workflows.
- Automated tests covered: generating a document from a template and verifying its content; editing and saving a clause; exporting a document and confirming the download.

Code Coverage

Jest Coverage Reports:

- Jest generates code coverage reports to ensure all critical components and functions are tested.
- The reports help identify untested parts of the code and improve test coverage.

Testing Goals:

- Achieve 80%+ test coverage, focusing on critical functionalities such as AI generation requests, clause editing, and export/e-signature flows.

Continuous Testing:

- Implement CI/CD pipelines to run tests automatically before deployment.
- Ensure new features do not break existing functionality.

KNOWN ISSUES

Despite extensive testing, the LegalEase application may have some known issues that could affect user experience. Below are identified issues along with possible causes and solutions.

Slow AI Generation Response

Issue:

- Document drafts take longer to generate, especially for lengthy or complex templates.

Possible Cause:

- AI provider rate limits or network latency.

Potential Solution:

- Implement caching for recently generated drafts.
- Show a loading spinner and progress indicator while waiting for the AI response.

Search Function Case Sensitivity

Issue:

- The search feature does not return results if queries don't match exact case.

Possible Cause:

- The search logic is case-sensitive and does not normalize input.

Potential Solution:

- Convert both user input and template names to lowercase before comparison.

UI Overlapping on Mobile Screens

Issue:

- Some UI components (e.g., navbar, editor toolbar) overlap on smaller screens.

Possible Cause:

- Missing proper media queries for responsive design.

Potential Solution:

- Adjust CSS breakpoints for better mobile compatibility.
- Use flexbox and grid layouts for dynamic positioning.

Incorrect Clause Substitution

Issue:

- Selecting a new template sometimes displays clauses from a previously generated document.

Possible Cause:

- The active document state is not resetting correctly between templates.

Potential Solution:

- Debug state management in DocumentContext.js to ensure proper resets.
- Implement a useEffect() hook to trigger a state reset when the template changes.

No Feedback on Failed AI Calls

Issue:

- When the AI generation request fails, users don't receive an error message.

Possible Cause:

- No error handling implemented around the AI API calls.

Potential Solution:

- Display clear error messages when generation requests fail.
- Implement try-catch blocks and show a fallback UI with a retry option.

These issues will be addressed in future updates to enhance the performance, usability, and reliability of the LegalEase application.

FUTURE ENHANCEMENTS

The LegalEase application is designed to provide a seamless experience for legal document creation and management. Below are potential future enhancements to improve usability, features, and overall performance.

User Authentication & Profiles

Enhancement:

- Implement user sign-up and login functionality.
- Allow users to save documents and drafts to their profiles.

Benefits:

- A personalized experience where users can organize and revisit their documents.

Clause Suggestion Engine

Enhancement:

- Suggest alternative or missing clauses based on document type and jurisdiction.
- Flag potentially risky or non-standard language for review.

Benefits:

- Reduces the risk of incomplete or inconsistent legal documents.

E-Signature & Workflow Integration

Enhancement:

- Deeper integration with e-signature providers for multi-party signing.
- Notifications and status tracking for documents awaiting signature.

Benefits:

- Streamlines the path from drafting to a fully executed document.

Dark Mode Support

Enhancement:

- Implement a dark mode toggle for better user experience.

Benefits:

- Reduces eye strain, especially during long editing sessions.

Multi-Language Document Generation

Enhancement:

- Enable users to generate documents in multiple languages.
- Add jurisdiction-aware clause libraries for different regions.

Benefits:

- Expands the application's usefulness to a broader, international user base.

Offline Mode for Saved Documents

Enhancement:

- Allow users to save documents for offline access and review.

Benefits:

- Improves usability in low-connectivity environments.